# Communication Efficient Gradient Coding: A Survey

Jay Mehta: 22B1281
Kshitij Vaidya: 22B1829
Tanay Bhat: 22B3303

May 6, 2026

# Contents

# Chapter 1

# Introduction

Distributed learning helps address the challenges faced by large-scale machine learning tasks by outsourcing its computations to worker nodes. Such a system is susecptible to communication bottenecks caused by end-to-end delays between the workers and the central server. Another possible issue is the presence of stragglers (nodes that are slow to respond). In this scenario, the central server does not have enough data to compute the gradient to update its model parameters. One way to mitigate this is to split the dataset into $k$ smaller batches and assign $d$ batches to each of the $n$ workers. The $i$-th worker then sends a function $f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d})$ of the gradients to the central server, where $g_{i_j}$ is the gradient computed by the worker on the $j$-th batch assigned to it. The server is able to recover the full gradient even if some of the workers are stragglers, as long as it receives enough $h_i$'s. We shall now formally define the problem of gradient coding and then discuss a communication efficient scheme to solve it.

## 1.1 Problem Statement

Consider a dataset $D = \{(x_i, y_i)\}_{i=1}^N$ where $x_i \in \mathbb{R}^l, y_i \in \mathbb{R}$. Our goal is to minimize the loss function $L(D; w) = \sum_{i=1}^N L(w; x_i, y_i)$ where $w \in \mathbb{R}^d$ is the model parameter. One way to do this to perform:

$$w^{(t+1)} = h(w^{(t)}, g^{(t)}) \tag{1.1}$$

Where $g^{(t)} = \sum_{i=1}^N \nabla L(w^{(t)}; x_i, y_i)$ and $h$ is some gradient based update. In a distributed setting, we assume $n$ workers $W_1, W_2, \ldots, W_n$ and split the dataaset $D$ into $k$ equal batches $D_1, D_2, \ldots, D_k$. Each worker is assigned $d$ batches and computes the following:

$$g_i^{(t)} = \sum_{(x,y)\in D_i} \nabla L(w^{(t)}; x, y) \tag{1.2}$$

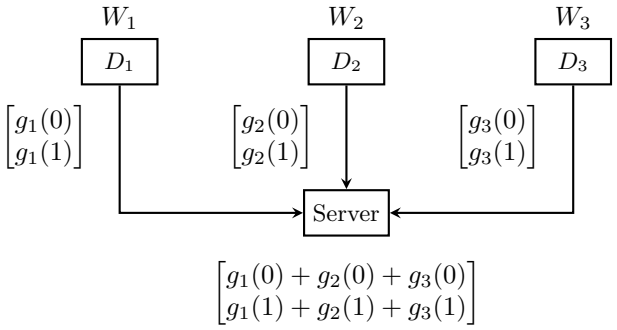
Hence we see that $g^{(t)} = \sum_{i=1}^k g_i^{(t)}$. We assume that $s$ worker nodes are stragglers and hence we only have access to $n - s$ of the $g_i^{(t)}$'s. Say for each$i \in 1, 2, \ldots n$ we represent the datasets assigned to $W_i$ as $\{D_{i_1}, D_{i_2}, \ldots, D_{i_d}\}$. The $i$-th worker then sends a function $f_i(g_{i_1}^{(t)}, g_{i_2}^{(t)}, \ldots, g_{i_d}^{(t)})$ of the partial gradients. This paper only considers linear functions of the form:

$$f_i(g_{i_1}^{(t)}, g_{i_2}^{(t)}, \ldots, g_{i_d}^{(t)}) = \sum_{j=1}^d a_{i_j} g_{i_j}^{(t)} \tag{1.3}$$

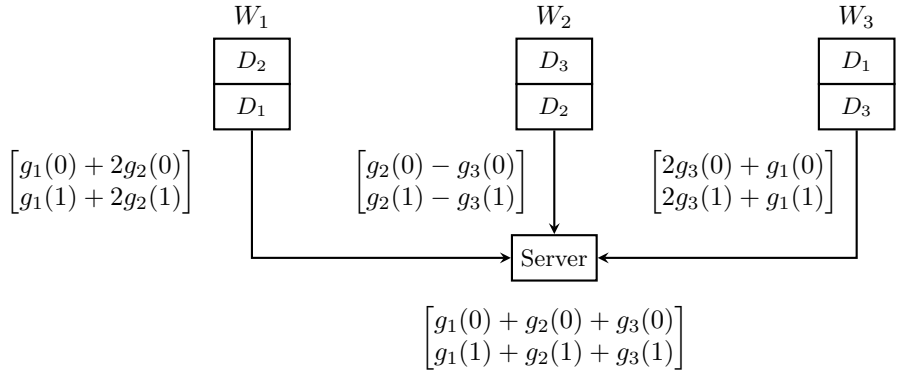
The partial gradient vector $g_i^{(t)} = [g_i^{(t)}(0), g_i^{(t)}(1), \ldots, g_i^{(t)}(l-1)]$. We can further generalize $f_i$ to be a linear combination of the coordinates of the partial gradients. This may require a large value of $l$ to be tolerant to stragglers or use a smaller $l$ but have less tolerance to stragglers. We shall drop the superscript $(t)$ for simplicity.

## 1.2 Tolerance-Efficiency Tradeoff

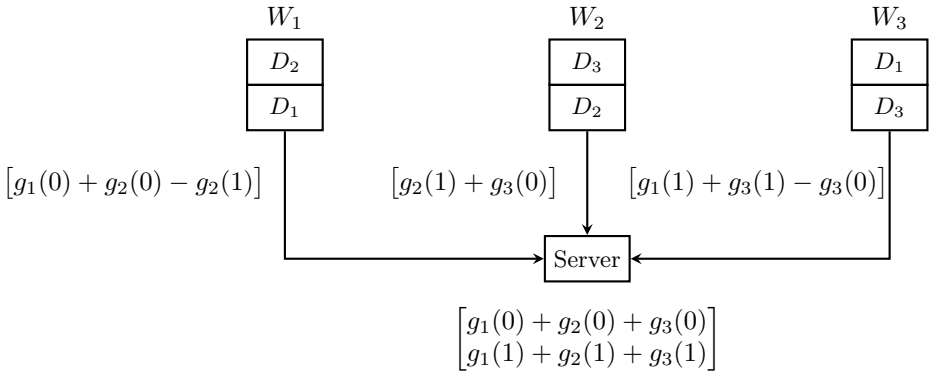
Consider the scenario where $n = 3, k = 3$ we can have the following assignment of batches to workers:




This is a basic scheme and the server needs to wait for all of the workers to respond before it can compute the full gradient. Such a scheme is neither communication efficient nor straggler tolerant. Now consider the following assignment of batches to workers:




Here the server can compute the full gradient even if one of the workers is a straggler. If $W_1$ is a straggler, the server can compute the full gradient as $g = f_1 - f_2$, if $W_2$ is a straggler, the server can compute the full gradient as $g = f_2 + f_3$ and if $W_3$ is a straggler, the server can compute the full gradient as $g = \frac{1}{2}(f_3 + f_1)$. However, this scheme is not communication efficient as each worker sends a vector of size $l$. Consider:




In this case $l = 1$ and the server computes the full gradient as $g(0) = f_1 + f_2$ and $g(1) = f_2 + f_3$. However, this scheme is not straggler tolerant as the server needs to wait for all the workers to respond before it can compute the full gradient. Hence we see that there is a tradeoff between straggler tolerance and communication efficiency. We shall now proceed with the main result presented by the paper and then discuss the scheme that achieves it.

# Chapter 2

# Main Theorem

## 2.1 Main Result

**Theorem 1.** *Given integers $n, k$ a triple $(d, s, m)$ where $1 \leq d \leq k$ and $m \geq 1$ is achievable if there is a distributed synchronous gradient coding scheme with $n$ workers and $k$ data batches such that:*

1. *Each worker is assigned $d$ data batches*
2. *There are $n$ functions $f_i : \mathbb{R}^{dl} \to \mathbb{R}^{l/m}$ such that the server can recover the full gradient $g$ from any $n - s$ of the vectors $f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d})$ where $i_1, i_2, \ldots, i_d$ are the indices of the data batches assigned to worker $W_i$.*
3. *Each $f_i$ is a linear combination of the coordinates of the partial gradients $g_{i_1}, \ldots, g_{i_d}$*

*and such a triple is achievable if and only if the following condition holds:*

$$\frac{d}{k} \geq \frac{s+m}{n} \tag{2.1}$$

We shall first show that any achievable $(d, s, m)$ must satisfy the above condition and then present the paper's coding scheme for $n = k$ that satisfies $d \geq s + m$ with equality.

## 2.2 Forward Direction

We assume that the triple $(d, s, m)$ is achievable and show that it must satisfy the condition $\frac{d}{k} \geq \frac{s+m}{n}$. Before wew proceed, we prove the following claim:

**Claim 1.** *For each $i \in [k]$, data batch $D_i$ is assigned to at least $s + m$ workers.*

*Proof.* Say for some $j \in [k]$ data batch $D_j$ is assigned to $a < s + m$ workers. WLOG assume these workers are $W_1, W_2, \ldots, W_a$ and suppose that first $s$ of these workers are stragglers. Based on our definition of achievability, the server should be able to recover the full gradient $g = g_1 + \ldots + g_k$ from the vectors $f_{s+1}, f_{s+2}, \ldots, f_n$.

As we can clearly calculate $g_j$ using the vectors $\{g_i : i \in [k] \setminus \{j\}\} \cup \{g_1 + \ldots + g_k\}$, we can see that $g_j$ can be obtained using the vectors $\{g_i : i \in [k] \setminus \{j\}\} \cup \{f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d}) : s + 1 \leq i \leq n\}$. Note that $D_j$ is only assigned to the first $a$ workers and $\{f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d}) : a + 1 \leq i \leq n\}$ can be otained using values in the set $\{g_i : i \in [k] \setminus \{j\}\}$. Hence $g_j$ can be obtained using only the vectors $\{g_i : i \in [k] \setminus \{j\}\} \cup \{f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d}) : s + 1 \leq i \leq a\}$.

Note that all $f_i$'s are linear functions of coordinates of the partial gradients and using the fact that $g_j$ is an $l$-dimensional vector, we see that the set $\{f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d}) : s + 1 \leq i \leq a\}$ must contain atleast $l$ linear combinations of the coordinates of $g_j$. Since $\{f_i(g_{i_1}, g_{i_2}, \ldots, g_{i_d}) : s + 1 \leq i \leq a\}$ contains elements that are $l/m$-dimensional, we see that we can have atmost $\frac{l}{m}(a - s)$ linear combinations of the coordinates

of $g_j$. Hence we must have $\frac{l}{m}(a-s) \geq l$ which implies that $a \geq s+m$. This is a contradicton and hence the claim is proved. ■

We shall now uuse this to prove the forward direction of the main theorem.

*Proof.* From the claim we see that each data batch is assigned to atleast $s+m$ workers hence in total there must be atleast $k(s+m)$ data subsets assigned over all the workers. Since each of the $n$ workers is assigned $d$ data batches, there are a total of $nd$ data subsets assigned over all the workers. Hence we must have $nd \geq k(s+m)$ which implies that $\frac{d}{k} \geq \frac{s+m}{n}$. ■

## 2.3 Reverse Direction

To prove the reverse direction, we construct a coding scheme for $n = k$ such that $d = s + m$ is achievable with equality.

*Proof.* We construct an explicit coding scheme that achieves $d = s + m$ with equality in the following steps:

1. Define polynomials $p_i(x)$ that encode which data subsets are assigned to each worker.
2. Construct the encoding matrix $B$ recursively using polynomial coefficients.
3. Show that any $(n - s)$ workers can jointly recover the full gradient vector.

**Notation** Let $\theta_1, \theta_2, \ldots, \theta_n$ be $n$ distinct real numbers (called "evaluation points"). We will use these to construct polynomials and a Vandermonde-like matrix for decoding.
**Key parameters:**

- $n$: number of workers
- $k = n$: number of data subsets (by assumption)
- $d = s + m$: computation load per worker
- $s$: number of stragglers tolerated
- $m$: communication reduction factor
- $l$: dimension of gradient vector (assumed divisible by $m$)

**Polynomial Construction** For each data subset $D_i$ (where $i \in [n]$), we define a polynomial $p_i(x)$ of degree $n - d$:

$$p_i(x) = \prod_{j=1}^{n-d} (x - \theta_{i \oplus j}), \tag{2.2}$$

where $\oplus$ denotes cyclic addition modulo $n$, i.e., $a \oplus b = a + b \pmod{n}$.
The polynomial $p_i(x)$ encodes batch assignment:

- The roots of $p_i(x)$ are $\theta_{i\oplus 1}, \theta_{i\oplus 2}, \ldots, \theta_{i\oplus(n-d)}$.
- Data subset $D_i$ is not assigned to workers $W_{i\oplus 1}, W_{i\oplus 2}, \ldots, W_{i\oplus(n-d)}$.
- Therefore, $D_i$ is assigned to workers where $p_i(\theta_j) \neq 0$.
- This gives exactly $d = s + m$ workers per batch (by cyclic symmetry).

Write $p_i(x)$ in standard form:

$$p_i(x) = \sum_{j=0}^{n-d} p_{i,j} x^j, \tag{2.3}$$

where $p_{i,j}$ are the coefficients. We can see that $\deg(p_i) = n - d$ and the leading coefficient $p_{i,n-d} = 1$.

**Recursive Polynomial Family** To handle the communication reduction factor $m$, we define a family of $m$ polynomials for each data subset. For each $i \in [n]$, define:

$$\begin{aligned} p_i^{(1)}(x) &:= p_i(x), \\ p_i^{(u)}(x) &:= x \cdot p_i^{(u-1)}(x) - p_{i,n-d-1}^{(u-1)} \cdot p_i^{(1)}(x), \quad u = 2, 3, \ldots, m. \end{aligned} \tag{2.4}$$

where $p_{i,j}^{(u)}$ denotes the coefficient of $x^j$ in $p_i^{(u)}(x)$. This recursive construction ensures:

- Each $p_i^{(u)}(x)$ has degree $n - d + u - 1$.
- The leading coefficient is always 1: $p_{i,n-d+u-1}^{(u)} = 1$.
- The last $m$ coefficients align properly for reconstruction:

$$p_{i,j}^{(u)} = 0 \quad \text{for } j \in \{n-d, n-d+1, \ldots, n-d+u-2\} \text{ when } u \geq 2.$$

- $p_i | p_i^{(u)}$ for all $u$, meaning:

$$p_i^{(u)}(\theta_{i\oplus 1}) = p_i^{(u)}(\theta_{i\oplus 2}) = \cdots = p_i^{(u)}(\theta_{i\oplus(n-d)}) = 0.$$

**Construction of Encoding Matrix $B$** Define an $(mn) \times (n-s)$ matrix $B = (b_{ij})$ with entries:

$$b_{(i-1)m+u,j} = p_{i,j-1}^{(u)} \quad \text{for all } i \in [n], u \in [m], j \in \{1, 2, \ldots, n-s\}. \tag{2.5}$$

The rows of $B$ are formed by stacking the coefficient vectors of the polynomials $p_1^{(1)}, \ldots, p_1^{(m)}, p_2^{(1)}, \ldots, p_n^{(m)}$ in order. For any $x \in \mathbb{R}$, the following matrix-vector product holds:

$$B \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^{n-s-1} \end{bmatrix} = \begin{bmatrix} p_1^{(1)}(x) \\ p_1^{(2)}(x) \\ \vdots \\ p_1^{(m)}(x) \\ p_2^{(1)}(x) \\ \vdots \\ p_n^{(m)}(x) \end{bmatrix}. \tag{2.6}$$

This directly follows from the polynomial representation and definition of $B$. The last columns of $B$ form a block diagonal matrix:

$$B_{[(n-d+1):(n-s)]} = \begin{bmatrix} I_m \\ I_m \\ \vdots \\ I_m \end{bmatrix}, \tag{2.7}$$

where $I_m$ is the $m \times m$ identity matrix, and there are $n$ blocks of $I_m$ (one for each worker). This property is crucial for recovery, it guarantees that any $(n-s) \times (n-s)$ submatrix of the Vandermonde matrix at columns corresponding to non-straggling workers will allow us to reconstruct the full gradient.

**Gradient Vector Partitioning** We partition the gradient vector to align with the communication reduction factor. For each $v \in \{0, 1, \ldots, l/m - 1\}$ and $j \in [n]$, define an $m$-dimensional sub-vector:

$$y_v^{(j)} := \begin{bmatrix} g_j(vm) \\ g_j(vm+1) \\ \vdots \\ g_j(vm+m-1) \end{bmatrix} \in \mathbb{R}^m. \tag{2.8}$$

For each $v$, stack these into an $(mn)$-dimensional vector:

$$z_v := \begin{bmatrix} y_v^{(1)} \\ y_v^{(2)} \\ \vdots \\ y_v^{(n)} \end{bmatrix} \in \mathbb{R}^{mn}. \tag{2.9}$$

This partitioning allows for:

1. Reducing transmission from dimension $l$ to dimension $l/m$ (communication reduction by factor $m$).
2. Applying the encoding matrix $B$ simultaneously to all $l/m$ blocks.
3. Reconstructing each block independently using the same Vandermonde inversion.

**Encoding at Workers** For worker $W_i$, compute the transmitted vector:

$$f_i(g_i, g_{i\oplus 1}, \ldots, g_{i\oplus(d-1)}) := \begin{bmatrix} z_0 \\ z_1 \\ \vdots \\ z_{l/m-1} \end{bmatrix} B \begin{bmatrix} 1 \\ \theta_i \\ \theta_i^2 \\ \vdots \\ \theta_i^{n-s-1} \end{bmatrix}. \tag{2.10}$$

This is an $(l/m)$-dimensional vector (reduced from dimension $l$). We have:

$$p_i^{(u)}(\theta_{i\oplus 1}) = p_i^{(u)}(\theta_{i\oplus 2}) = \cdots = p_i^{(u)}(\theta_{i\oplus(n-d)}) = 0.$$

Substituting into the encoding, the product $z_v B \begin{bmatrix} 1 & \theta_i & \cdots & \theta_i^{n-s-1} \end{bmatrix}^T$ depends only on:

- The sub-vectors $y_v^{(j)}$ for $j$ such that $p_j(\theta_i) \neq 0$, i.e., for $j \in \{i, i\oplus 1, \ldots, i\oplus(d-1)\}$.
- Thus, worker $W_i$ only needs access to data subsets $D_i, D_{i\oplus 1}, \ldots, D_{i\oplus(d-1)}$.

**Decoding at Master Node** Without loss of generality, assume the non-straggling workers are $\{1, 2, \ldots, n-s\}$. Define the $(n-s)\times(n-s)$ Vandermonde matrix:

$$A := \begin{bmatrix} 1 & 1 & \cdots & 1 \\ \theta_1 & \theta_2 & \cdots & \theta_{n-s} \\ \theta_1^2 & \theta_2^2 & \cdots & \theta_{n-s}^2 \\ \vdots & \vdots & \ddots & \vdots \\ \theta_1^{n-s-1} & \theta_2^{n-s-1} & \cdots & \theta_{n-s}^{n-s-1} \end{bmatrix}. \tag{2.11}$$

This matrix is invertible (standard property of Vandermonde matrices with distinct evaluation points). From the received vectors $f_1, f_2, \ldots, f_{n-s}$, the master obtains:

$$z_v BA \quad \text{for all } v \in \{0, 1, \ldots, l/m-1\}. \tag{2.12}$$

To recover $z_v B e_{n-d+u}$ (which corresponds to the $u$-th component of the sum gradient at position $vm+u-1$), multiply by $A^{-1}e_{n-d+u}$:

$$z_v B e_{n-d+u} = z_v BAA^{-1} e_{n-d+u}. \tag{2.13}$$

By the block diagonal structure of $B$ (equation 15), we have:

$$z_v B e_{n-d+u} = \sum_{j=1}^{n} g_j(vm+u-1) \quad \text{for all } v \in \{0, 1, \ldots, l/m-1\}, u \in [m]. \tag{2.14}$$

Combining over all $v$ and $u$:

$$\{z_v B e_{n-d+u} : v \in \{0, \ldots, l/m - 1\}, u \in [m]\} = \left\{ \sum_{j=1}^{n} g_j(i) : i \in \{0, 1, \ldots, l-1\} \right\} = g^{(1)} + g^{(2)} + \cdots + g^{(n)}. \tag{2.15}$$

Thus, the full gradient vector is recovered. We have shown that the coding scheme satisfies all required conditions, completing the achievability proof.

■

## 2.4 Coding Scheme